

Python for ML- NumPy, Pandas and Matplotlib

Hannah Chen



Agenda - NumPy

- What is NumPy
- Why use NumPy
- Creation in NumPy
- Describing a Numpy Array
- Reshaping in NumPy
- Indexing and Slicing in NumPy
- Broadcasting in NumPy
- More Functions in Numpy

Agenda - Pandas

- What is Pandas
- Why use Pandas
- Data Structures in Pandas
- Filtering a Data Frame
- Handling Missing Values with Pandas
- Sorting in a Data Frame
- Data Manipulation with Pandas
- Data Analysis with Pandas
- Combining DataFrames with Pandas

Agenda - Matplotlib

- What is Matplotlib
- Why use Matplotlib
- Basic plot types in Matplotlib
- Basic plots in Machine Learning

What is NumPy (Numerical Python)

- NumPy is a Python library that provides support for *large, multi-dimensional* arrays and matrices.
- NumPy offers a large collection of high-level *mathematical* functions to operate on these arrays.

\$1	\$2	\$5	\$6	\$7	\$8	\$2
-----	-----	-----	-----	-----	-----	-----

\$1	\$2	\$5
\$6	\$7	\$8
\$1	\$2	\$5
\$1	\$2	\$5

38	28	11			
451	467	468			
726	631	539			

What is NumPy (Numerical Python)

- NumPy is a Python library that provides support for *large, multi-dimensional* arrays and matrices.
- NumPy offers a large collection of high-level *mathematical* functions to operate on these arrays.



Why Use Numpy (Numerical Python)

- **Speed:** NumPy is faster than regular Python lists.
- **Functionality:** It provides a lot of built-in functions that make mathematical calculations easy.
- **Efficiency:** With NumPy, you can perform operations on entire lists of data without having to use loops.

Why Use Numpy (Numerical Python)

- **Speed:** NumPy is faster than regular Python lists.



Vs.



Why Use Numpy (Numerical Python)

- **Speed:** NumPy is faster than regular Python lists.
- **Functionality:** It provides a lot of built-in functions that make mathematical calculations easy.
- **Efficiency:** With NumPy, you can perform operations on entire lists of data without having to use loops.

Why Use Numpy (Numerical Python)

- **Functionality:** It provides a lot of built-in functions that make mathematical calculations easy.



Vs.



Why Use Numpy (Numerical Python)

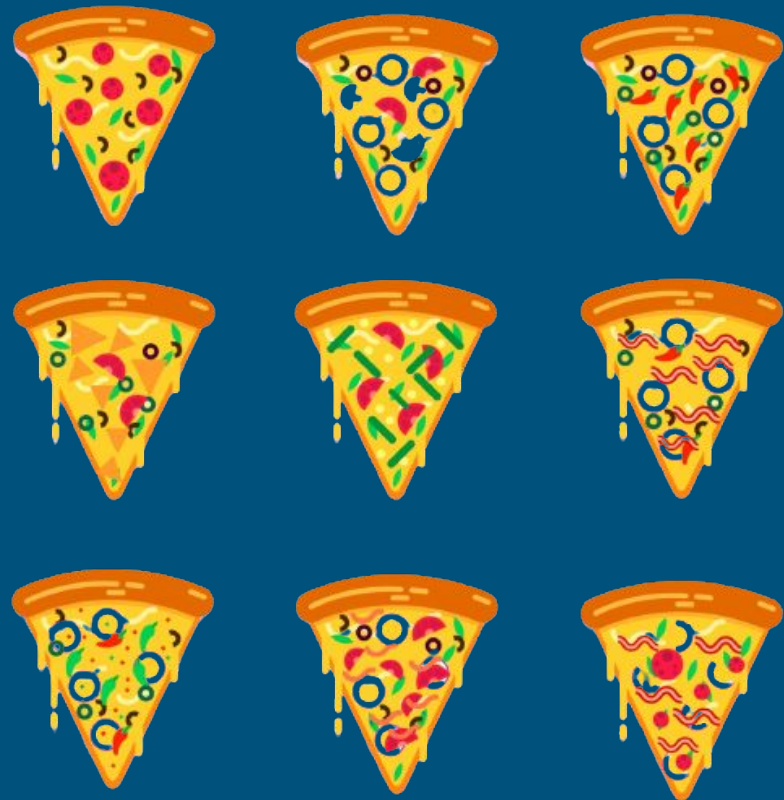
- **Speed:** NumPy is faster than regular Python lists.
- **Functionality:** It provides a lot of built-in functions that make mathematical calculations easy.
- **Efficiency:** With NumPy, you can perform operations on entire datasets without having to use loops.

Why Use Numpy (Numerical Python)

- **Efficiency:** With NumPy, you can perform operations on entire datasets without having to use loops.



Vs.



Creation in Numpy — 1-d array and 2-d array

index	0	1	2	3	4	5	6
	3	3	5	7	2	3	5

1-d array

		column index						
		0	1	2	3	4	5	6
row index	0	3	3	5	7	2	3	5
	1	0	2	0	0	2	4	5
	2	7	6	6	2	3	3	6

2-d array

Creation in Numpy — 1-d array

state	account_ length	international _plan	total_intl _calls	number_vmail_ messages	number_customer_ service_calls	churn
OH	107	no	3	26	1	no
NJ	137	no	5	0	0	no
OH	84	yes	7	0	2	no
OK	75	yes	3	0	3	no
MA	121	no	7	24	3	no
MO	147	yes	6	0	0	no
LA	117	no	4	0	1	no
WV	141	yes	5	37	0	no
IN	65	no	6	0	4	yes

Creation in Numpy — 2-d array

state	account_ length	international _plan	total_intl _calls	number_vmail_ messages	number_customer_ service_calls	churn
OH	107	no	3	26	1	no
NJ	137	no	5	0	0	no
OH	84	yes	7	0	2	no
OK	75	yes	3	0	3	no
MA	121	no	7	24	3	no
MO	147	yes	6	0	0	no
LA	117	no	4	0	1	no
WV	141	yes	5	37	0	no
IN	65	no	6	0	4	yes

Describing a Numpy Array

- **shape:** returns a tuple representing the dimensions of a NumPy array. Each element of the tuple indicates the size of the array along a particular dimension.
- **size:** # of elements in a Numpy array.
- **ndim:** # of dimensions

Reshaping in Numpy

- Reshaping enables us to reconstruct an array while keeping the same data but altering its arrangement in different shape.
- The only requirement is that the both original and the reshaped array must have the same number of elements (i.e., the same size).

Reshaping in Numpy

1	2	5
6	7	8
1	2	5
1	2	5



- Reshape to 12 rows and 1 column
- Reshape to 3 rows and 4 columns
- Reshape to 6 rows and 2 columns
- Reshape to 2 rows and 6 columns
- Reshape to 4 rows and 10 columns

? X ? Matrix

Reshaping in Numpy

1	2	5
6	7	8
1	2	5
1	2	5

4 X 3 Matrix



Reshape to 12 rows and 1 column



Reshape to 3 rows and 4 columns



Reshape to 6 rows and 2 columns



Reshape to 2 rows and 6 columns

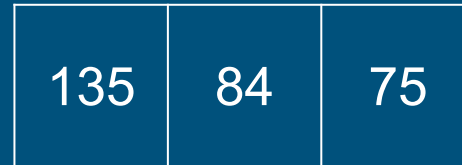
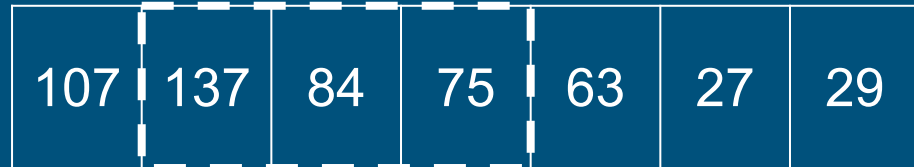
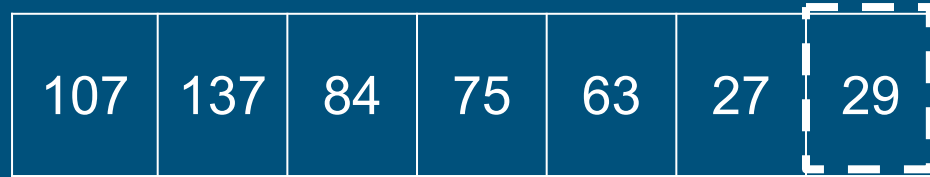
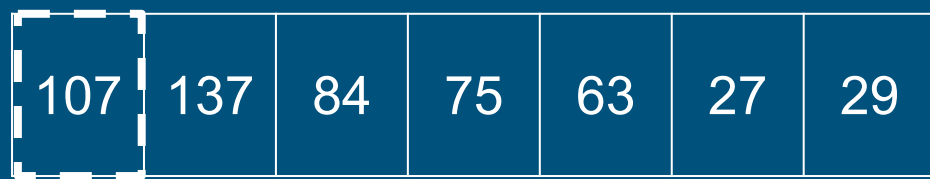


Reshape to 4 rows and 10 columns

Indexing and Slicing in Numpy

- **Indexing:** retrieves a single element.
- **Slicing:** retrieves a range of elements.
- Indexing and slicing enable us to extract the relevant features or samples for analysis or model training.

Indexing and Slicing in Numpy — 1-d array



Indexing and Slicing in Numpy — 1-d array



Which elements > 30 & < 100 ?



Indexing and Slicing in Numpy — 2-d array

26	1	4
0	0	6
0	2	8
0	3	2



26

26	1	4
0	0	6
0	2	8
0	3	2



2

Indexing and Slicing in Numpy — 2-d array

26	1	4
0	0	6
0	2	8
0	3	2



0	2	8
---	---	---

26	1	4
0	0	6
0	2	8
0	3	2



1
0
2
3

Indexing and Slicing in Numpy — 2-d array

26	1	4
0	0	6
0	2	8
0	3	2



2	8
3	2

26	1	4
0	0	6
0	2	8
0	3	2

Which elements
are odd
numbers?



1	3
---	---

Broadcasting in Numpy - What is Broadcasting in NumPy?

- Broadcasting is a powerful feature in NumPy. It describes how NumPy treats arrays with different shapes during arithmetic operations.
- Subject to certain constraints, the smaller array is “broadcast” across the larger array so that they have compatible shapes.
-
- This broadcasting technique simplifies array operations and makes them more efficient by avoiding the need for explicit loops.

Broadcasting in Numpy - example 1

Coding Challenge: Write a Python function “add_ten”

```
def add_ten(input_arr):
```

26	1	4
0	0	6
0	2	8
0	3	2



36	11	14
10	10	16
10	12	18
10	13	12

Broadcasting in Numpy - example 1

Coding Challenge: Write a Python function “add_ten”

26	1	4
0	0	6
0	2	8
0	3	2

+ 10 =

26	1	4
0	0	6
0	2	8
0	3	2

+

10	10	10
10	10	10
10	10	10
10	10	10

=

36	11	14
10	10	16
10	12	18
10	13	12

Broadcasting in Numpy - example 2

26	1	4
0	0	6
0	2	8
0	3	2



1	10	100
---	----	-----



Broadcasting in Numpy - example 2

26	1	4
0	0	6
0	2	8
0	3	2



1	10	100
---	----	-----

=

?

26	1	4
0	0	6
0	2	8
0	3	2



1	10	100
1	10	100
1	10	100
1	10	100



=

26	10	400
0	0	600
0	20	800
0	30	200

Broadcasting in Numpy - example 3

26	1	4
0	0	6
0	2	8
0	3	2



1
3
2
2



Broadcasting in Numpy - example 3

26	1	4
0	0	6
0	2	8
0	3	2



1
3
2
2



26	1	4
0	0	6
0	2	8
0	3	2



1	1	1
3	3	3
2	2	2
2	2	2



26	1	4
0	0	2
0	1	4
0	1.5	1

Broadcasting in Numpy - example 4

26	1	4
0	0	6
0	2	8
0	3	2

+

1	2	3
4	5	6
7	8	9
10	11	12

= ?

Broadcasting in Numpy - example 4

26	1	4
0	0	6
0	2	8
0	3	2

+

1	2	3
4	5	6
7	8	9
10	11	12

= ?

26	1	4
0	0	6
0	2	8
0	3	2

+

1	2	3
4	5	6
7	8	9
10	11	12

=

27	3	7
4	5	12
7	10	17
10	14	14

Broadcasting in Numpy - General rules

- When operating on two arrays, NumPy compares their shapes *element-wise*. It starts with the trailing dimensions and works its way forward.
- Two arrays A, B are compatible when:
 - one of them is the singular value
 - one is $R * C$ and other is $1 * C$
 - one is $R * C$ and other is $R * 1$
 - they have same shape

More Functions in Numpy - Array Manipulation

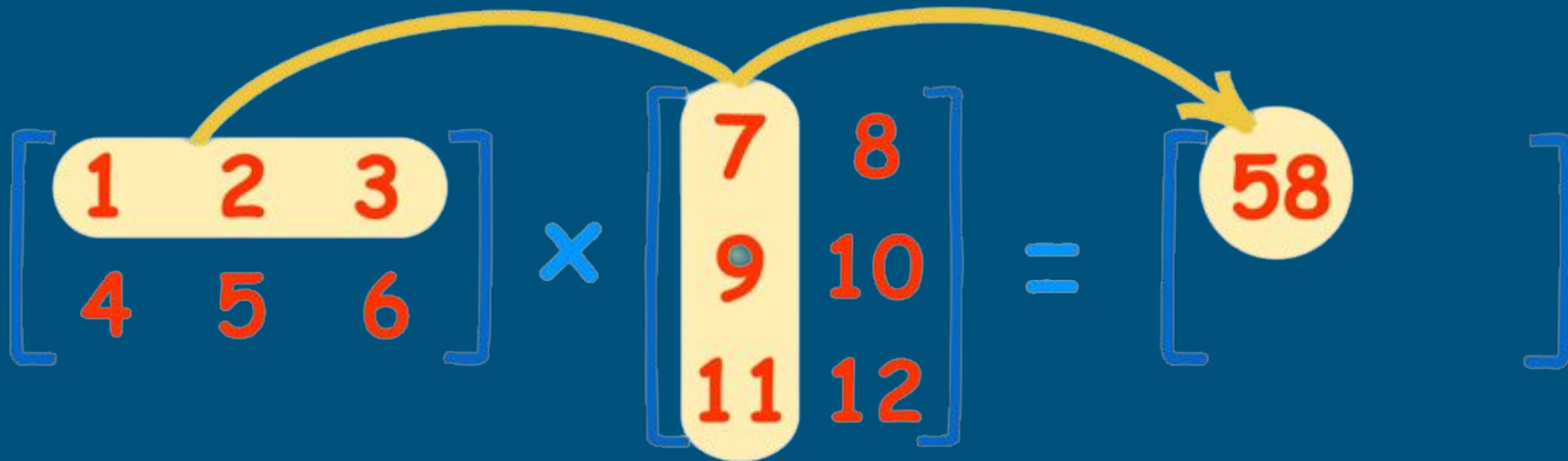
- `np.diagonal()`
- `np.transpose(matrix), matrix.T`
- `np.hstack()`
- `np.vstack()`
- `np.concatenate()`
- `np.append()`

More Functions in Numpy - Arithmetic Operation

- `np.min()`, `np.max()`
- `np.argmin()`, `np.argmax()`
- `np.unravel_index()`
- `np.mean()`, `np.std()`
- `np.cumsum()`
- `np.trace()`
- `np.count_nonzero()`
- `np.inner()`
- `np.cross()`
- `np.dot()`
- `np.matmul()`
- `np.multiply()`

Frequently Used Functions in Numpy - np.matmul()

“Matrix Multiplication”



$$1 * 7 + 2 * 9 + 3 * 11 = 58$$

Frequently Used Functions in Numpy - np.matmul()

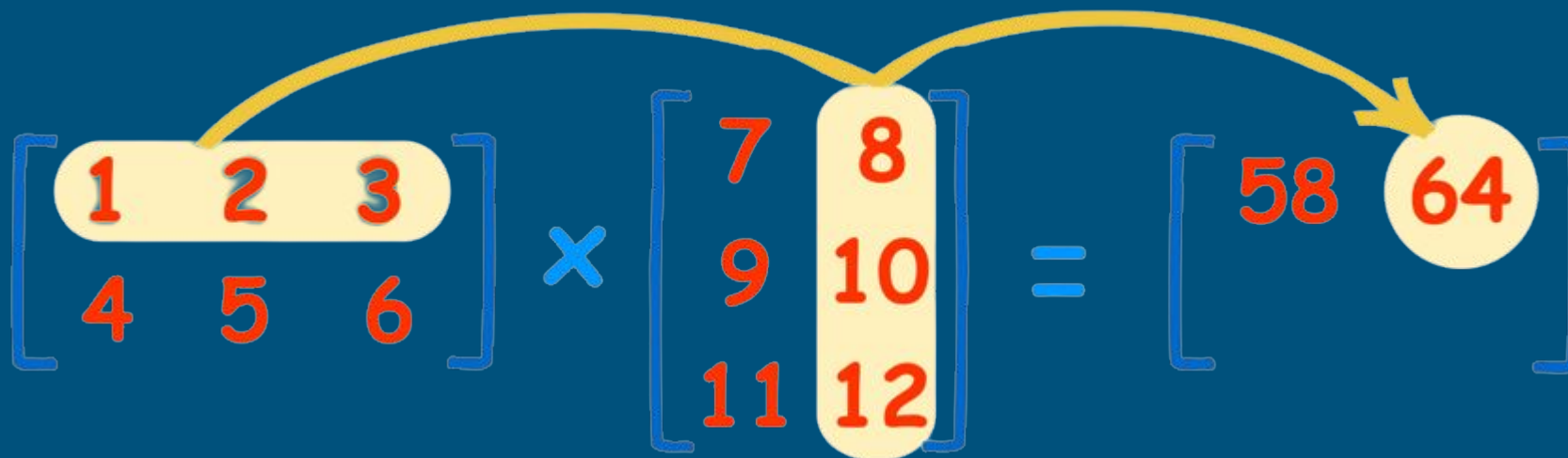
“Matrix Multiplication”

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ 32 & 37 \end{bmatrix}$$

$$1 * 8 + 2 * 10 + 3 * 12 = 64$$

Frequently Used Functions in Numpy - np.matmul() Rule

“Matrix Multiplication”


$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ \end{bmatrix}$$

When we do Matrix Multiplication :

- The number of columns of the 1st matrix must equal the number of rows of the 2nd matrix.
- And the result will have the same number of rows as the 1st matrix, and the same number of columns as the 2nd matrix.

More Functions in Numpy - Other

- `np.where()`
- `np.sort()`

Agenda - Pandas

- What is Pandas
- Why use Pandas
- Data Structures in Pandas
- Filtering a Data Frame
- Handling Missing Values with Pandas
- Sorting in a Data Frame
- Data Manipulation with Pandas
- Data Analysis with Pandas
- Combining DataFrames with Pandas

What is Pandas (comes from Panel Data)

- Pandas is a powerful Python library widely used for *data manipulation* and *analysis*.
- Pandas offers data structures and functions that simplify working with structured data, making it an essential tool in data science and analysis

Why use Pandas

- Using Pandas offers several benefits for data manipulation and data analysis:
 - Efficient Data Handling
 - Data Alignment
 - Handling Missing Data
 - Data Integration
 - Flexible Data Transformation
 - Integration with Other Libraries

Data Structures in Pandas

- Pandas **Series** is a one-dimensional array with axis labels.
- Pandas **DataFrame** is two-dimensional data structure with labeled rows and columns.

Series A		Series B		DataFrame		
	apples		oranges		apples	oranges
0	3	0	0		0	3
1	2	1	3		1	2
2	0	2	7		2	0
3	1	3	2		3	1

Series and DataFrame Creation in Pandas

Time to Code!

Filtering a Data Frame - Why do we need filtering?

- When we need to remove redundant or unnecessary data for some tasks
- When we need to find customers eligible for a promotion
- When we need to filter out rows or columns have missing values
- When we want to filter customers based on the amount spent

Filtering a Data Frame

- Filtering with ``loc`` and ``iloc`` methods
- Filtering by Selecting a Subset of Columns
- Filtering by condition(s)

Filtering a Data Frame - `loc` and `iloc` methods

- **loc** uses row and column labels.
- **iloc** uses row and column indexes.

	column index		0	1	2
row index	label		Name	Age	Education
0	March	2			
1	April	3			
2	May	4			
3	June	5			
4	July	6			

Filtering a Data Frame - Selecting a Subset of Columns

- Select a single column
- Select multiple columns

	state	account_length	international_plan	total_intl_calls	churn
0					
1					
2					

Select multiple columns

Select a column

Filtering a Data Frame - Filtering by condition(s)

- Filtering by a condition
- Filtering by multiple conditions

The operators we can use in filtering by condition(s):

- ==: equal
- !=: not equal
- >: greater than
- >=: greater than or equal to
- <: less than
- <=: less than or equal to
- & : and
- | : or

Handling Missing Values with Pandas

- Missing Value Types and Representation
- Finding the Missing Values
- Dropping Rows and Columns with Missing Values
- Replacing the Missing Values

Handling Missing Values with Pandas

- Missing values are essentially data we don't have
- NaN : stands for “Not a Number”
 - Python's **None**
 - **np.nan**



Finding the Missing Values

- `isna()` function: returns **True** to indicate a missing value
- `notna()` function: returns **False** for missing values

	state	account_length	churn
0	CA	314	yes
1	CO	?	no
2	?	44	yes

Dropping Rows and Columns with Missing Values

- `dropna()` function:
 - ``axis``: 0 (default value, indicates row), 1
 - ``how``: “any” (default value), “all”
 - ``thresh``: a row or column that at least X non-missing values will be kept, otherwise will be dropped.

	state	account_length	churn
0	CA	314	?
1	CO	?	?
2	?	44	?

Replacing the Missing Values

- Dropping may not be the best option in many cases
- Replace missing values with:
 - the average value of the column
 - the most frequent value in the column
 - a random number
- `fillna()` function

	state	account_length	churn
0	CA	314	Yes
1	CO	100	Yes
2	CO	44	no

Sorting in a Data Frame

- Sort DataFrames based on their column features
- `sort_values()` : sort a DataFrame by one or more of its columns
 - First argument: either a column label or a list of column labels
 - ``ascending`` : ascending (default value), descending

	Name	age
0	David	33
1	Emily	20
2	Emma	44



	Name	age
0	Emily	20
1	David	33
2	Emma	44

Data Manipulation with Pandas

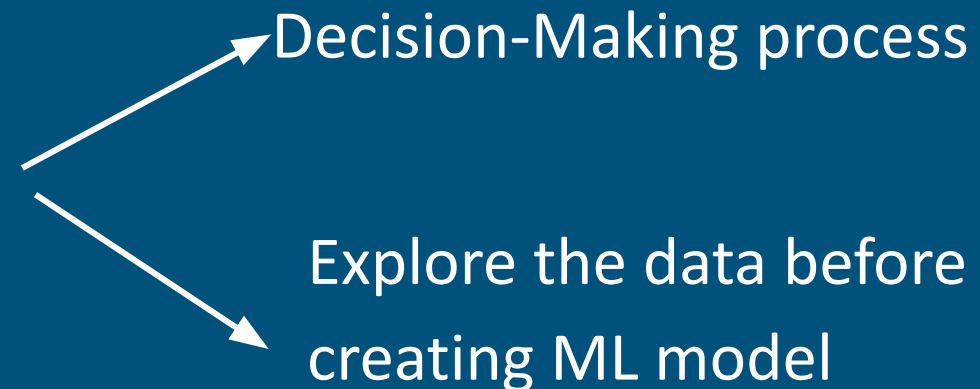
- `drop()`: Remove rows or columns from a DataFrame
- `replace()`: Replace values within a DataFrame
- `apply()`: Applying a given function along an axis of a DataFrame or Series.
- Add a new column from current data

	user_id	account_length	total_orders	total_amt
0	1232as	31	4	156
1	1323wv	0	2	127
2	134pdf	0	3	135
3	342t6ff	23	4	466

Data Analysis with Pandas

- Data Analysis: The process of inferring insights, discovering useful information, and drawing results from the data
 - `groupby()`: groups rows of a DataFrame based on one or more columns
 - `agg()`: aggregate data within groups created by `groupby()`
 - `reset_index()`: reset the index of a DataFrame
 - `rename()`: change the labels (names) of rows or columns in a DataFrame
 - `idxmax()`: Return index of first occurrence of maximum over requested axis

	total_order	age
0	20	33
1	40	20
2	100	44



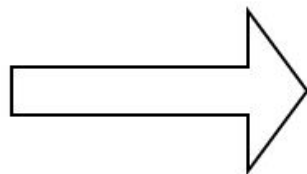
Data Analysis with Pandas - groupby(), agg()

- The `groupby` function creates the groups, but they don't provide any information unless we do some aggregations.

DataFrame

product_group	product_code	price
A	1001	9
A	1002	14
B	1101	21
A	1003	12
B	1104	19
C	1201	7
B	1105	25

groupby("product_group")



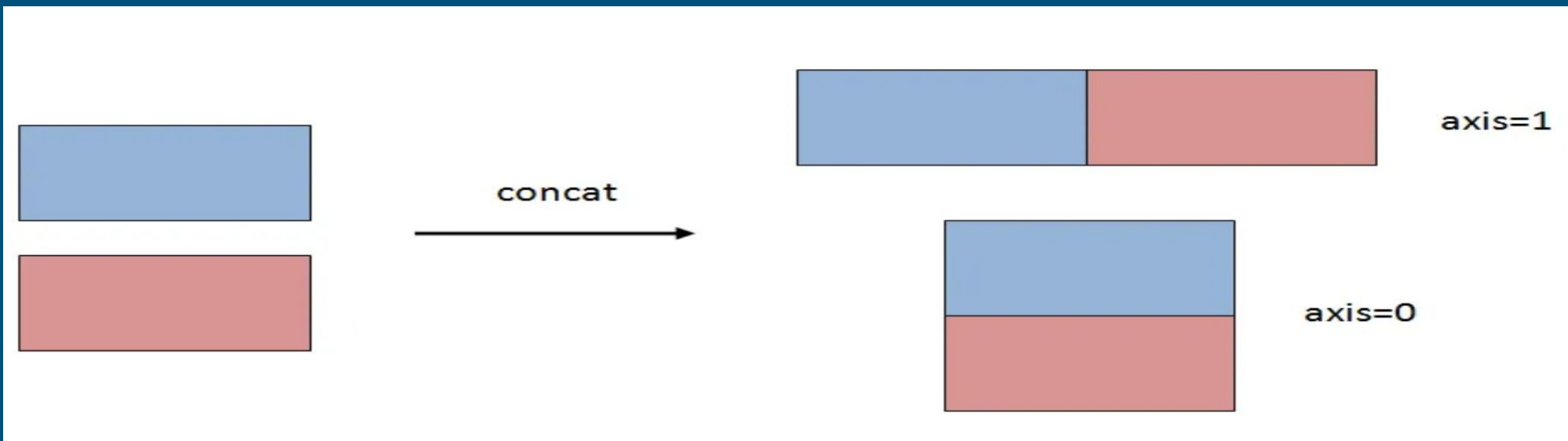
product_group	product_code	price
A	1001	9
A	1002	14
A	1003	12

product_group	product_code	price
B	1101	21
B	1104	19
B	1105	25

product_group	product_code	price
C	1201	7

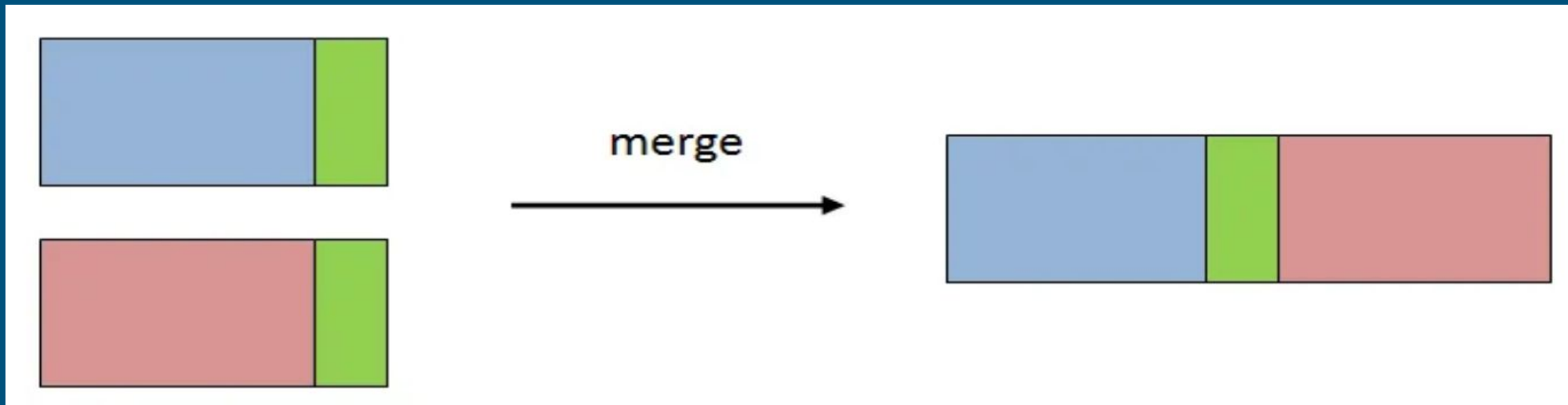
Combining DataFrames with Pandas - The concat function

- Concatenation is the process of combining DataFrames along a particular axis (either vertically or horizontally).
- When you concatenate DataFrames, you're essentially stacking them on top of each other (vertically) or side by side (horizontally).
- The resulting DataFrame maintains the original column names and indexes of the individual DataFrames.



Combining DataFrames with Pandas - The merge function

- Merging is the process of combining DataFrames based on common columns or indexes.
- It is often used to combine DataFrames with related information, similar to how SQL JOIN operation works.
- Merging allows us to specify which columns to use as keys for merging and how to handle overlapping column names.



Agenda - Matplotlib

- What is Matplotlib
- Why use Matplotlib
- Basic plot types in Matplotlib
- Basic plots in Machine Learning

What is Matplotlib

- Matplotlib is a popular Python library for data visualization and plotting.
- It provides a wide range of tools for creating static, animated, and interactive plots.

Why Use Matplotlib

- Matplotlib is easy to use and integrates well with other Python libraries like NumPy and Pandas.
- It supports a variety of plot types, making it suitable for various data visualization tasks.
- Matplotlib offers complete control over plot customization, allowing you to create publication-quality graphics.
- `matplotlib.pyplot`

Basic plot types in Matplotlib

- Line Plot
- Scatter Plot
- Bar Chart
- Histogram
- Pie Chart
- Heatmap
- Bubble Chart

Basic plots in Machine Learning

- Box Plot
- Correlation Heatmap
- Confusion Matrix
- ROC(Receiver operating characteristic) Curve
- Precision-Recall Curve
- Learning Curve
- Feature Importance Plot